

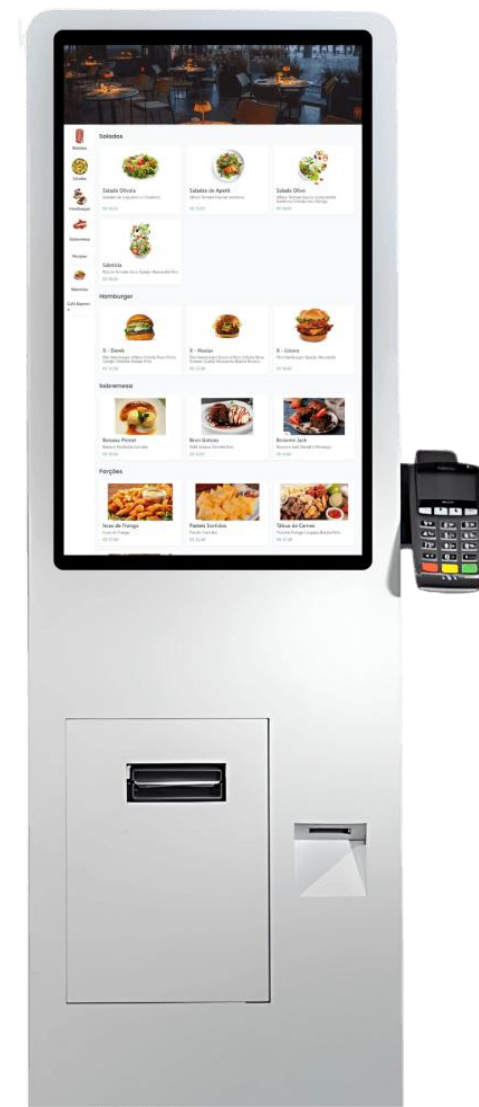


Projeto Final

Josué Schwartzhaupt

21/06/2026

Disciplina: Programação 1 (Prof. Rodrigo Goulart)



Objetivo e modelagem do projeto

Objetivo

Desenvolver um sistema de autoatendimento para lanchonetes utilizando Java e JavaFX, permitindo que clientes realizem pedidos de forma autônoma e que o estabelecimento acompanhe o andamento dos atendimentos.

Modelagem

- **Interface:** interação com o usuário.
- **Controle:** gerenciamento dos pedidos.
- **Modelo:** representação dos dados e regras de negócio.

Principais Classes

- Sistema
- Pedido
- Item (abstrata)
- Lanche
- Bebida
- Sobremesa
- Cardapio
- TelaGeralController

O Aplicativo (requisitos funcionais)

Área do Cliente

- Visualização do cardápio
- Adição e remoção de itens
- Alteração de tamanho e sabor
- Visualização do valor total
- Confirmação do pedido

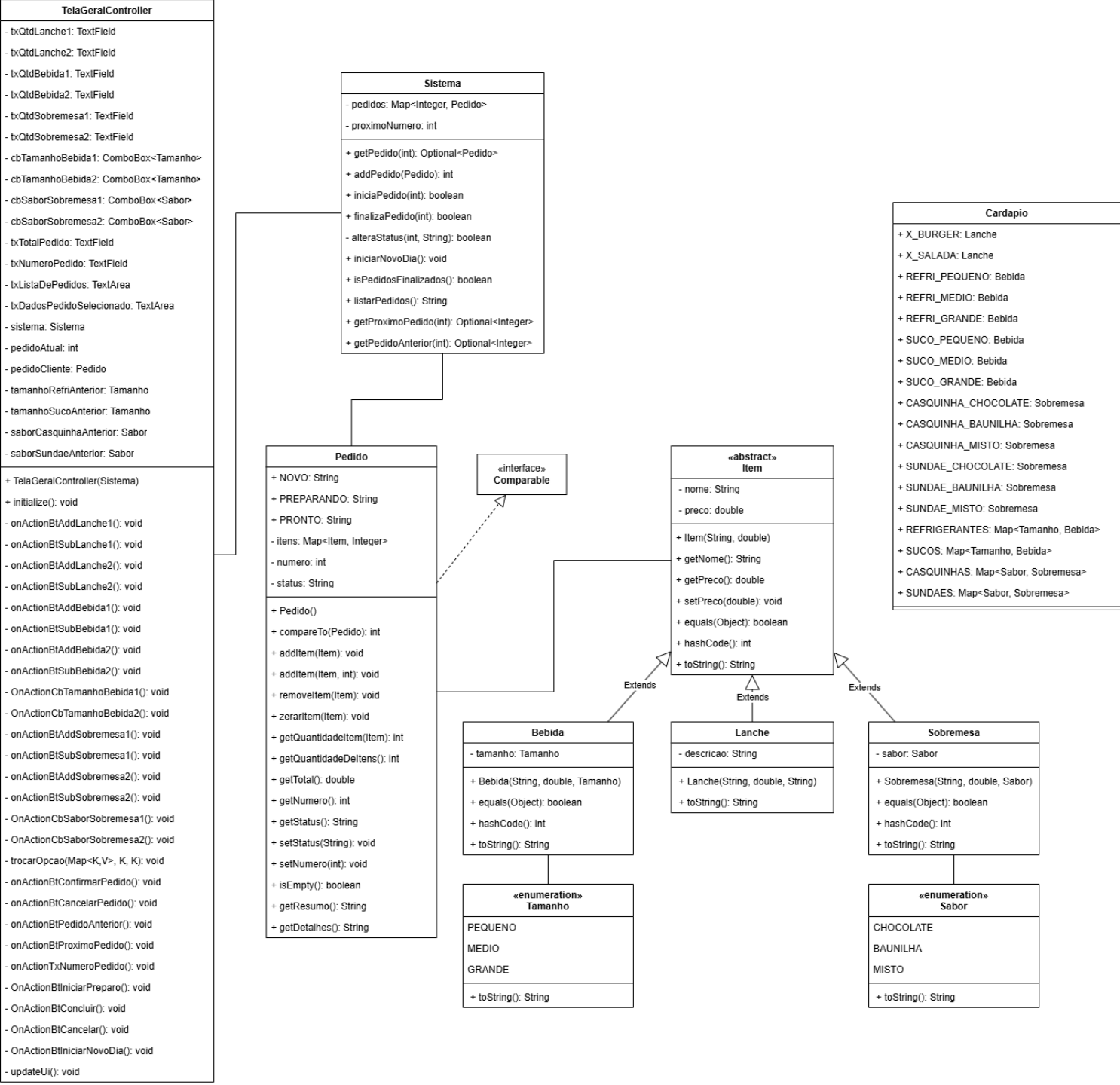
Área do Estabelecimento

- Visualização dos pedidos
- Navegação entre atendimentos
- Controle de status
- A Preparar
- Preparando
- Pronto
- Início de um novo dia

O Aplicativo (requisitos funcionais)

The screenshot shows a web application for ordering food. On the left, a vertical sidebar contains three tabs: 'Lanches' (selected), 'Bebidas', and 'Sobremesas'. The main area displays two burger images. Each image has a quantity selector below it, consisting of a minus button, a text box with '0', and a plus button. At the bottom left, the text 'Total:' is followed by a text box containing '0,00' and two buttons: 'Confirmar' and 'Cancelar'. On the right side, there is a section titled 'Lista de Pedidos:' containing a text box with the text 'Pedido #001 | 1 item | R\$ 18,00 | A Preparar'. Below this is a section titled 'Processar Pedido' with a text box labeled 'Nº:' and two buttons: 'Anterior' and 'Proximo'. Underneath is a section titled 'Dados:' with a large empty text box. At the bottom right, there are three buttons: 'Iniciar Preparo', 'Concluir', and 'Cancelar', and a button labeled 'Iniciar Novo Dia'.

O Aplicativo (requisitos técnicos)



Encapsulamento

- Atributos privados
- Métodos getters e setters

Herança

- Item → Lanche, Bebida e Sobremesa

Polimorfismo

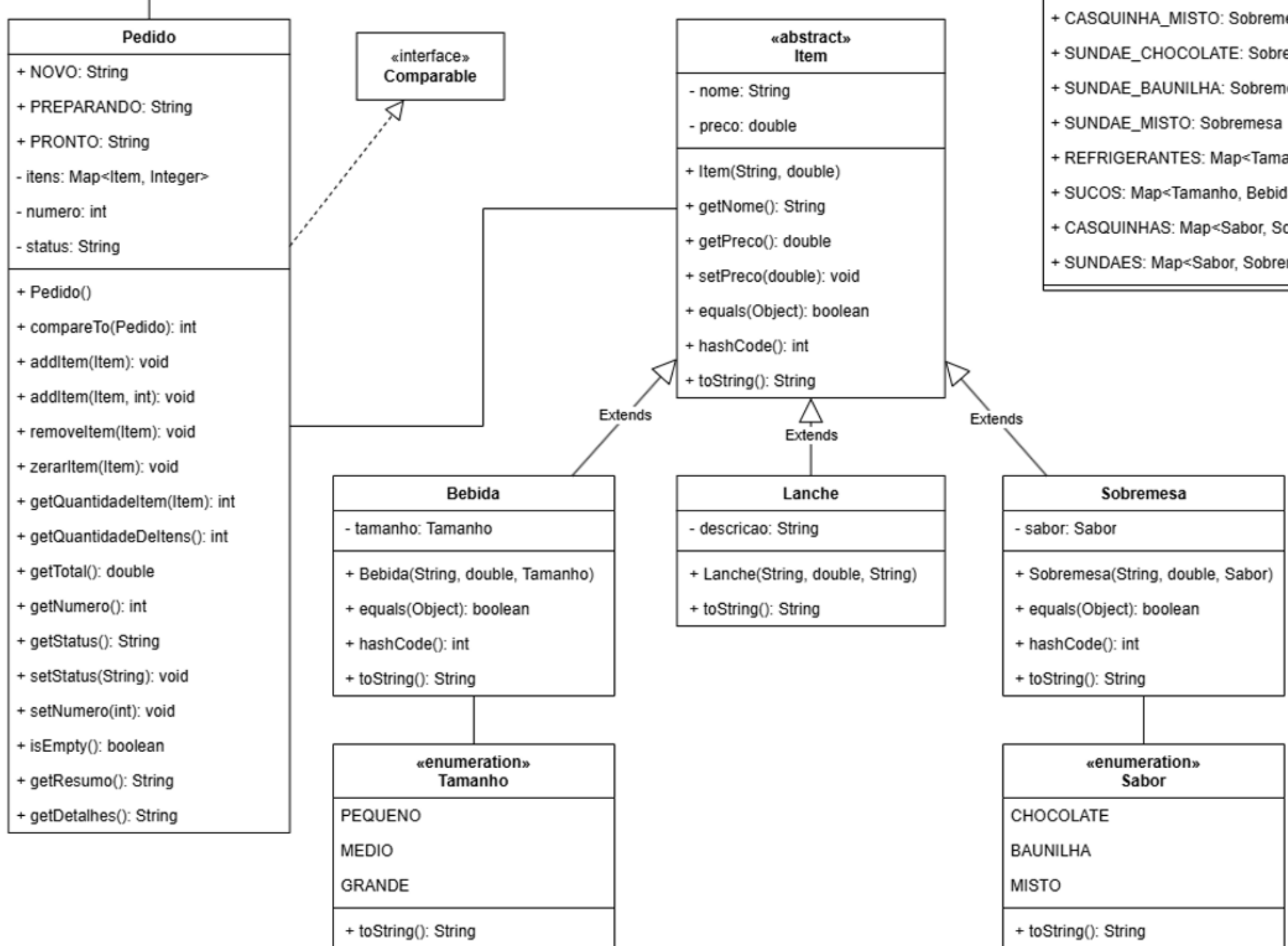
- Manipulação dos produtos através da classe Item

Abstração

- Classe abstrata Item

Interfaces

- Comparable<Pedido>



Coleções

- Map<Integer, Pedido>
- Map<Item, Integer>

Recursos do Java

- Optional
- Stream API
- Generics
- equals() e hashCode()

TelaGeralController	
- txQtdLanche1: TextField - txQtdLanche2: TextField - txQtdBebida1: TextField - txQtdBebida2: TextField - txQtdSobremesa1: TextField - txQtdSobremesa2: TextField - cbTamanhoBebida1: ComboBox<Tamanho> - cbTamanhoBebida2: ComboBox<Tamanho> - cbSaborSobremesa1: ComboBox<Sabor> - cbSaborSobremesa2: ComboBox<Sabor> - txTotalPedido: TextField - txNumeroPedido: TextField - txListaDePedidos: TextArea - txDadosPedidoSelecionado: TextArea - sistema: Sistema - pedidoAtual: int - pedidoCliente: Pedido - tamanhoRefriAnterior: Tamanho - tamanhoSucoAnterior: Tamanho - saborCasquinhaAnterior: Sabor - saborSundaeAnterior: Sabor	- onActionBtAddLanche1(): void - onActionBtSubLanche1(): void - onActionBtAddLanche2(): void - onActionBtSubLanche2(): void - onActionBtAddBebida1(): void - onActionBtSubBebida1(): void - onActionBtAddBebida2(): void - onActionBtSubBebida2(): void - OnActionCbTamanhoBebida1(): void - OnActionCbTamanhoBebida2(): void - onActionBtAddSobremesa1(): void - onActionBtSubSobremesa1(): void - onActionBtAddSobremesa2(): void - onActionBtSubSobremesa2(): void - OnActionCbSaborSobremesa1(): void - OnActionCbSaborSobremesa2(): void - trocarOpcao(Map<K,V>, K, K): void - onActionBtConfirmarPedido(): void - onActionBtCancelarPedido(): void - onActionBtPedidoAnterior(): void - onActionBtProximoPedido(): void - onActionTxNumeroPedido(): void - OnActionBtIniciarPreparo(): void - OnActionBtConcluir(): void - OnActionBtCancelar(): void - OnActionBtIniciarNovoDia(): void - updateUi(): void
+ TelaGeralController(Sistema) + initialize(): void	

Sistema
- pedidos: Map<Integer, Pedido> - proximoNumero: int
+ getPedido(int): Optional<Pedido> + addPedido(Pedido): int + iniciaPedido(int): boolean + finalizaPedido(int): boolean - alteraStatus(int, String): boolean + iniciarNovoDia(): void + isPedidosFinalizados(): boolean + listarPedidos(): String + getProximoPedido(int): Optional<Integer> + getPedidoAnterior(int): Optional<Integer>

Limitações e trabalhos futuros

- Sistema de carrinho separado do pedido final
- Simulação de meios de pagamento
- Geração de planilhas com as vendas do dia
- Persistência dos dados em banco de dados